Software Engineering Lab

Lab 38

Mutation Testing

Mengungkap Ilusi Keamanan 100% Code Coverage dengan Fault Injection Berbasis AST



Lukman / lukman-ss

Ilusi Keamanan Code Coverage

Cakupan baris mengukur eksekusi, bukan verifikasi kebenaran logika.

METRIK TRADISIONAL

100% line coverage artinya setiap baris dieksekusi minimal sekali.
Tidak ada asersi substantif: test hanya memastikan fungsi berjalan tanpa error.
Contoh: assert result > 0 pada kalkulasi diskon yang seharusnya == expected.

DAMPAK PRODUKSI

Operator relasional salah: >= berubah menjadi > pada threshold diskon.
Logika boolean terbalik: || menjadi && mengubah kualifikasi free shipping.
Bug logika lolos ke production karena test suite hijau penuh.

DATA LAB 38

discount_weak_test.go mencapai 100% statement coverage pada discount.go.
Namun mutation score-nya 0.00%: 0 dari 15 mutant terdeteksi.
Test suite lemah menutupi seluruh baris tanpa menangkap fault injection.

Code coverage tradisional sering disalahartikan sebagai tolok ukur kualitas pengujian. Metrik ini hanya menjawab pertanyaan apakah sebuah pernyataan dalam sumber pernah dieksekusi selama suite tes berjalan. Pernyataan tersebut bisa saja dieksekusi berkali-kali tanpa pernah memiliki asersi yang memverifikasi nilai keluaran, batasan logika, atau branch kondisional. Akibatnya, tim dapat merasa aman melihat CI berwarna hijau dengan cakupan 100%, padahal tidak ada pengujian yang sanggup mendeteksi kesalahan operator maupun pergeseran nilai batas.

Coverage measures execution, not verification. Zero killed mutants reveals the gap.

Mengapa Review Manual dan Assertion Naif Gagal

Inspeksi mata hanya menjangkau jalur yang dipikirkan developer.

BIAS KONFIRMASI

Developer cenderung menulis contoh input yang mendukung implementasi. Edge case acak seperti float precision, batas negatif, atau nol sering terlupa.

KEKURANGAN ASSERTION

Assert boolean `freeShipping` rarely checked.
Assert exact discount rate missing; only positivity validated.

SKALA LOGIKA

Satu fungsi `CalculateDiscount` mengandung 8 operator relasional dan 4 boolean.
Kombinasi threshold 1000, 500, 100, 200 serta ItemCount > 2 menciptakan lintasan kompleks.

Pemeriksaan manual atau penulisan assertion seadanya tidak cukup karena batas kognitif engineer. Manusia memikirkan skenario paling umum; kondisi pinggir jarang terdeteksi tanpa eksplorasi sistematis. Pada modul diskon ini, logika bercabang meliputi tier VIP, premium, dan standard dengan ambang jumlah item serta kupon. Tanpa generator fault injection, kombinasi operator yang mungkin meleset tetap tersisa sebagai blind spot yang hanya terbongkar saat mutan berhasil bertahan.

Human testing samples expectations; mutation testing probes violations.

Mutation Testing vs Code Coverage

Coverage memeriksa apa yang dieksekusi; mutation memeriksa apa yang diverifikasi.

ANALOGI ROTI DAN SELAI

Coverage melaporkan berapa persen roti terlapisi selai.
Mutation memeriksa apakah selai itu benar cokelat atau zat lain.
Roti tertutup penuh bisa jadi selainya salah total.

RUMUS SKOR

Mutation Score = (Killed / Total) × 100%
Killed = mutant terdeteksi oleh test suite yang gagal.
Survived = mutant lolos karena assertion lemah atau input tidak relevan.

HIPIOTESIS FUNDAMENTAL

Competent Programmer Hypothesis: bug biasanya operator/tanda kecil.
Coupling Effect: fault sederhana bertingkat menjadi kegagalan kompleks.

Model mental mutation testing memisahkan konsep eksekusi dari verifikasi. Code coverageibarat laporan roti yang diolesi selai: yang diukur hanya luas permukaan yang tertutup. Mutation testing menambahkan lapisan skeptisisme dengan menanyakan kandungan selai itu. Jika logika bisnis diubah sedikit—misalnya tanda pembanding atau operator aritmetika—dan test suite masih lolos, maka selai tersebut bukan yang seharusnya. Skor mutasi memberi angka objektif atas kedalaman asersi.

Execute everything, verify nothing is equal to passing everything.

Pipeline Mesin Mutation Testing

Source -> AST Parse -> Mutant Plans -> Isolated Execution -> Score.

LANGKAH PIPELINE

Read discount.go -> parser.ParseFile dengan go/ast.
ast.Inspect collect BinaryExpr dan BasicLit nodes.
Build MutationPlan dengan Apply closure dan undo revert.

EKSEKUSI PARALEL

Runner.Run meluncurkan goroutine per rencana mutasi.
Tiap goroutine membuat fresh token.FileSet untuk isolasi total.
Hasil ditulis ke slice berbasis indeks tanpa mutex.

OUTPUT LAPORAN

Report aggregating Killed, Survived, Equivalent counts.
Skor dihitung akhir dengan formula persentase.
Breakdown per mutant menampilkan line, operator, dan status.

Pipeline ini dibangun murni menggunakan pustaka standar Go tanpa dependensi eksternal. Proses dimulai dengan pemindaian AST untuk mengenali node ekspresi biner dan literal konstanta. Setiap node yang terdeteksi dikemas menjadi rencana mutasi yang menyertakan fungsi transformasi serta fungsi undo. Runner mengeksekusi rencana tersebut secara paralel dalam goroutine terisolasi, masing-masing bekerja dengan salinan AST baru sehingga tidak terjadi race condition antar mutan.

Parse once, mutate many, isolate always, aggregate deterministically.

AST Mutation pada Relational & Boolean Node

MutationPlan menyematkan Apply closure untuk transformasi dan undo reversibel.

```
case token.GEQ:
  plans = append(plans, MutationPlan{
    Apply: func(f *ast.File) func() {
      expr.Op = token.GTR
      return func() { expr.Op = token.GEQ }
    },
  })
```

```
case token.OR: // ||
  expr.Op = token.AND // &&
  undo restores token.OR
```

```
Multiply to divide: `*` -> `/`
Subtract to add: `-` -> `+`
Integer constant +1 shift at line 40
```

Generator mutasi menelusuri AST menggunakan `ast.Inspect`. Saat menemui ekspresi biner, ia memeriksa token operator dan menghasilkan rencana yang membalikkan relasional, boolean, atau aritmetika. Closure `Apply` melakukan modifikasi langsung pada node AST, sementara fungsi undo yang dikembalikan memulihkan state semula. Pendekatan ini menjamin setiap mutan dievaluasi dalam lingkungan AST bersih tanpa kontaminasi state goroutine lain.

Mutate AST in place, revert immediately, never share mutable nodes.

Failure Path: Operator `>=` Berubah Menjadi `>`

Threshold diskon standard jatuh tepat di batas 100.0.

```
else if order.TotalAmount >= 100.0 {
    rate = 0.05
```

```
else if order.TotalAmount > 100.0 {
    rate = 0.05
```

Kesalahan pergeseran threshold dari `>=` menjadi `>` tampak sepele, tetapi mengubah hasil diskon pelanggan dengan total belanja tepat 100.0. Test suite lemah hanya memeriksa apakah diskon tidak negatif; kondisi ini tetap terpenuhi karena rate berubah menjadi nol, bukan nilai negatif. Akibatnya, mutan bertahan dan pelanggan kehilangan potongan yang seharusnya mereka dapatkan. Skenario ini menggambarkan kegagalan propagasi nilai kritis ke asersi.

INPUT KRITIS

TotalAmount = 100.0, Tier = STANDARD
Asli: rate = 0.05; Mutan: rate = 0.0
Weak test hanya cek `res3.DiscountRate < 0` → tetap PASS

Inclusive vs exclusive boundaries hide regressions behind weak numeric guards.

Mekanisme Mutasi vs Gejala yang Dihasilkan

Empat kategori operator memunculkan pola kegagalan berbeda.

Operator	Mutasi	Gejala	Deteksi Weak Test
Relational (>=)	>= -> >	Boundary threshold shift	Missed (rate 0 still >= 0)
Relational (==)	== -> !=	Tier logic inversion	Missed (amount still positive)
Boolean (	)	-> &&	VIP free shipping disabled	Missed (amount positive)
Boolean (&&)	&& ->	Coupon boost applies wrongly	Missed (rate positive)
Arithmetic (*)	* -> /	Discount calculation exploded	Missed (final amount positive)
Arithmetic (-)	- -> +	Final amount inflated	Missed (value > 0)
Boundary (+1)	2 -> 3	Coupon requires 3 items	Missed (ItemCount 3 passes)

Setiap kategori mutasi menghasilkan gejala kegagalan yang spesifik. Operator relasional mengubah ambang batas kualifikasi tier dan diskon; boolean flip mengacaukan kondisi VIP dan kupon; aritmetika membalikkan kalkulasi potongan; pergeseran batas menggeser syarat minimum item. Test suite lemah consistently gagal karena semua gejala tersebut tetap menghasilkan nilai uang positif, sehingga asersi berbasis tanda tidak pernah memicu kegagalan.

Fault categories map to symptom families; weak assertions fall in the positivity trap.

Tiga Keputusan Kritis Saat Membangun Mutation Engine

Isolasi AST, strategi seleksi node, dan definisi kill condition.

KEPUTUSAN 1: ISOLASI

Pilih fresh `token.NewFileSet()` per goroutine.
Hindari mutex dengan penulisan berbasis indeks slice.
Gunakan closure undo untuk memulihkan AST setelah evaluasi.

KEPUTUSAN 2: SELEKSI NODE

Targetkan `*ast.BinaryExpr` untuk operator.
Targetkan `*ast.BasicLit` bertipe `token.INT` untuk batas.
Abaikan node string/float agar boundary shift fokus pada integer.

KEPUTUSAN 3: KILL CONDITION

TestFunc menerima sourc bytes mutan, bukan eksekusi binary.
Strong test membandingkan string AST secara eksak != original.
Weak test mensimulasikan heuristik assertion positif saja.

Konstruksi mesin mutation testing membutuhkan tiga keputusan arsitektural mendasar. IsolasiAST mencegah goroutine saling menimpa node operator. Seleksi node terbatas pada ekspresi biner dan literal integer memastikan fokus pada fault yang relevan secara bisnis. Finally, definisi kill condition menentukan apakah tes dianggap detektif: strong test menggunakan kesamaan teks AST, sedangkan weak test hanya meniru pemeriksaan tanda positif pada output keuangan.

Isolate state, target syntax, define kill by semantic divergence.

Bukti Eksekusi: Demo CLI Mutation Testing

Hasil running `go run ./cmd/demo` menunjukkan kontras skor mutasi.

OUTPUT DEMO

Target: internal/service/discount.go — 15 mutants generated
Weak suite kills: 0/15 mutants (0.00%)
Strong suite kills: 15/15 mutants (100.00%)
Race detector: `go test -race ./...` passed

PECAHAN MUTAN

[SURVIVED] Mutant 1–15: semua operator flips & boundary shifts lolos.
[KILLED] Mutant 1–15: strong test menangkap semua perubahan teks AST.

Demo CLI secara eksplisit menunjukkan kesenjangan antara cakupan statement dan deteksi fault. Enam belas baris output ringkas merangkum 15 mutasi yang dihasilkant dari file tunggal. Suite lemah tetap lulus semua mutasi karena asersi hanya memeriksa posivitas nilai. Sebaliknya, suite kuat mengevaluasi perbedaan teks AST secara utuh sehingga setiap mutasi menghasilkan string berbeda dan terdeteksi sebagai killed.

Numbers don't lie: 0% vs 100% mutation score from identical coverage.

YANG TIDAK DIBUKTIKAN

Laboratorium ini memiliki celah validasi yang perlu diketahui.

SKOR 100% BUKAN STANDAR INDUSTRI

Tidak ada ambang mutation score universal yang disepakati.
Target skor harus disesuaikan dengan profil risiko proyek.

EVALUASI AST TEXTUAL

Strong test menggunakan perbandingan string AST, bukan subprocess kompilasi.
Ini trade-off ringan untuk demonstrasi; production tool compile subprocess.

EQUIVALENT MUTANTS

Mutan semantik identik tidak dapat dibunuh secara teoretis.
Penanganan equivalent memerlukan heuristic atau scoped analysis.

STATEMENTDELETE TIDAK DIPAKAI

Enum `StatementDelete` dideklarasikan untuk klasifikasi lengkap.
Tidak diemisi karena target AST tidak memiliki void side-effect-free statement.

Meskipun demonstrasi ini menunjukkan kontras yang tajam, ada batasan penting yang tidak diselesaikan. Skor mutasi 100% bukanlah standar industri baku; tiap organisasi menetapkan ambang berdasarkan risiko domain. Evaluasi strong test di sini mengandalkan pencocokan teks AST, bukan kompilasi subprocess nyata, sebagai kompromi performa. Selain itu, equivalent mutants tetap menjadi persoalan yang secara matematis undecidable dan memerlukan filter heuristik dalam praktik.

Mutation testing reveals gaps; it does not replace risk-based quality gates.

Trade-off Arsitektur: Kecepatan vs Ketepatan Deteksi

Pilih antara evaluasi tekstual cepat dan kompilasi subprocess akurat.

AST TEXTUAL (LAB INI)

Kecepatan: evaluasi sub-milidetik per mutan.
Ketepatan: menangkap semua perubahan sintaks, tapi bukan semantic.
Risiko: false positive jika formatter mengubah whitespace.

SUBPROCESS COMPILE (PRODUKSI)

Kecepatan:Compilation overhead per mutant.
Ketepatan: semantic + syntax; false positive rendah.
Risiko: dependency coupling, environment lock.

REKOMENDASI HYBRID

Lintasan cepat: textual AST filter untuk PR sehari-hari.
Lintasan berat: subprocess compile untuk modul kritikal.

Trade-off utama terletak pada cara menentukan kill condition. Evaluasi berbasis teks AST menawarkan kecepatan tinggi dengan overhead minimal, cocok untuk pipeline CI harian. Namun, pendekatan ini rentan terhadap false positive akibat perbedaan formatting. Di sisi lain, kompilasi subprocess memberikan jaminan semantic yang lebih kuat, tetapi membutuhkan environment build yang stabil dan biaya waktu lebih besar. Solusi hibrida merekomendasikan penggunaan textual filtering sebagai gate cepat dan subprocess compile sebagai gate kritis.

Speed versus semantic certainty; hybrid gates balance both.

Goroutine -> Validate -> Write Result

Isolasi AST per mutan memastikan race-clean execution.

FLOW CONCURRENCY

Runner.launch loop `for i, plan := range plans`.
每 goroutine: `fset := token.NewFileSet()` + `parser.ParseFile`.
Apply mutasi, render source, jalankan `testFn`.
Tulis `results[idx]` langsung tanpa mutex.

ISOLASI STATE

Setiap goroutine memiliki salinan AST sendiri.
Closure undo mengembalikan node sebelum keluar goroutine.
Sinkronisasi hanya pada `sync.WaitGroup`.

VERIFIKASI RACE

`go test -race ./...` passed.
Tidak ada shared mutable node antar goroutine.

Desain konkurensi mesin ini mengutamakan isolasi state AST. Setiap mutan dieksekusi dalam goroutine terpisah yang membuka file set baru dan mem-parsing ulang sumber asli. Hasil evaluasi ditulis langsung ke slice yang dialokasikan sebelumnya menggunakan indeks plan, menghilangkan kebutuhan mutex. Pengembalian nilai menggunakan closure undo memastikan node AST tetap bersih setelah proses selesai. Race detector Go mengonfirmasi tidak ada data race.

Fresh parse per mutant, indexed writes, no shared mutable state.

Before vs After: Weak Assertion vs Strong Assertion

Perubahan asersi memindahkan mutation score dari 0% ke 100%.

BEFORE (WEAK)

Asersi: hanya `FinalAmount <= 0`, `DiscountTotal < 0`, `DiscountRate < 0`.
Coverage: 100% statement.
Mutation Score: 0/15 (0.00%).
Gejala: semua mutan survived karena output tetap positif.

AFTER (STRONG)

Asersi: exact value, boundary, boolean flag, coupon condition.
Coverage: 100% statement.
Mutation Score: 15/15 (100.00%).
Gejala: setiap perubahan operator/nilai langsung terdeteksi.

Perbandingan sebelum dan sesudah memperlihatkan bahwa cakupan 100% tidak berubah, namun mutation score melonjak drastis hanya dengan memperbaiki kualitas asersi. Test suite lemah mengandalkan pemeriksaan tanda positif yang longgar. Suite kuat menambahkan assertion eksak pada nilai akhir, rate diskon, flag free shipping, dan kondisi kupon. Perubahan minimal ini menutup seluruh celah deteksi dan membunuh semua mutan.

Same coverage, different assertions, dramatically different fault exposure.

Analisis Mendalam: Mengapa Mutan Tertentu Lolos

Mapping survival pattern ke kelemahan asersi spesifik.

MUTAN 1: || -> &&

Kode: `order.Tier == TierVIP || order.TotalAmount >= 1000.0`
Mutasi: hanya VIP dengan amount >= 1000 yang masuk.
Input weak test: VIP + 1200 -> masih masuk, finalAmount positif.
Weak assert tidak memeriksa freeShipping boolean.

MUTAN 9: > -> >=

Kode: `order.ItemCount > 2`
Mutasi: ItemCount == 3 tetap memenuhi, bahkan ItemCount == 2 juga.
Input weak test: ItemCount = 3 -> coupon boost tetap berlaku.
Tidak ada assert untuk rate boost eksak.

MUTAN 11: * -> /

Kode: `discountAmount := order.TotalAmount * rate`
Mutasi: perhitungan menjadi pembagian.
Input weak test: TotalAmount besar -> hasil bagi tetap positif.
Assert hanya memeriksa positive, bukan exact formula.

Analisis mendalam menunjukkan pola survival yang konsisten. Mutan Boolean flip lolos karena test tidak pernah mengevaluasi kondisi branch yang berubah. Mutan relasional lolos karena input berada jauh di atas atau bawah threshold, sehingga pergeseran batas tidak mengubah outcome positif. Mutan aritmetika lolos karena nilai numerik tetap berada dalam domain positif meskipun formula berubah. Pola ini menegaskan bahwa weak assertions tidak pernah melakukan propagasi nilai kritis ke asersi.

Survival patterns map directly to assertion blind spots.

Bukti Unit Test Engine dan Skoring

Engine unit test memverifikasi kategori mutasi dan perbedaan skor.

TEST ENGINE

TestEngine_GeneratesMutants memastikan semua 4 tipe mutasi hadir.
TestEngine_MutationScoreDifference memastikan weak < strong score.
TestDomainService_DirectCalculation memverifikasi calculation accuracy.

ASSERTION STRONG

VIP customer: DiscountRate == 0.20, FreeShipping == true.
Premium 500: DiscountRate == 0.10.
Standard 100: DiscountRate == 0.05.
Coupon +3 items: rate boost 0.05 applied.

HASIL

Semua test passing termasuk race detector.
Mutation score weak = 0%, strong = 100%.

Unit test engine memverifikasi dua hal mendasar: kelengkapan kategori mutasi dan perbedaan skor antara suite lemah dan kuat. Test strong secara eksplisit asserted nilai exact pada setiap kasus uji, termasuk boundary amount 99.99 dan 100.0 serta flag boolean free shipping. Keberhasilan seluruh test bersamaan dengan race detector menunjukkan implementasi konkuren aman dan asersi ketat efektif membunuh semua mutan.

Exact assertions are the only reliable mutant killers.

Breakdown Lengkap 15 Mutan dan Status Deteksi

Tabel detail setiap mutasi beserta penyebab survival/killed.

RELATIONAL (8)

M1: || -> && (line 31) SURVIVED weak / KILLED strong
M2: == -> != (line 31) SURVIVED weak / KILLED strong
M3: >= -> > (line 31) SURVIVED weak / KILLED strong
M4: && -> || (line 33) SURVIVED weak / KILLED strong
M5: == -> != (line 33) SURVIVED weak / KILLED strong
M6: >= -> > (line 33) SURVIVED weak / KILLED strong
M7: >= -> > (line 35) SURVIVED weak / KILLED strong
M8: == -> != (line 48) SURVIVED weak / KILLED strong

BOOLEAN (4)

M9: >= -> > (line 40) SURVIVED weak / KILLED strong
M10: && -> || (line 40) SURVIVED weak / KILLED strong
M11: * -> / (line 44) SURVIVED weak / KILLED strong
M12: - -> + (line 45) SURVIVED weak / KILLED strong

ARITHMETIC & BOUNDARY (3)

M13: || -> && (line 48) SURVIVED weak / KILLED strong
M14: == -> != (line 48) SURVIVED weak / KILLED strong
M15: >= -> > (line 48) SURVIVED weak / KILLED strong

Breakdown lengkap menunjukkan pola uniform: semua 15 mutan bertahan pada suite lemah dan terbunuh pada suite kuat. Tidak ada mutant yang equivalent dalam kasus ini karena setiap perubahan operator atau konstanta menghasilkan perilaku output yang berbeda secara semantik. Uniform survival pada weak suite menegaskan bahwa kelemahan terletak pada asersi, bukan pada ketidakefektifan generator mutasi.

Uniform survival points to assertion weakness, not generator failure.

Pertimbangan Performa dan Skalabilitas

Biaya komputasi mutation testing dan strategi mitigasi.

BIAYA KOMPUTASI

15 mutant x parse+render+test per mutant.
Isolasi AST menambah overhead parse berulang.
Runtime tetap sub-detik untuk modul berukuran kecil.

STRATEGI SKALA

Differential mutation: hanya mutasi file yang berubah di PR.
Incremental scoring: target skor minimum per modul.
Parallel goroutine mengurangi wall-clock time secara linear.

BATASAN PRODUKSI

Full mutation testing mahal untuk codebase raksasa.
Pertimbangkan focus pada domain core dan jalur kritikal.

Mutation testing memperkenalkan biaya komputasi tambahan karena setiap mutan memerlukan parsing ulang dan evaluasi. Namun, isolasi goroutine dan penulisan hasil berbasis indeks menjaga overhead tetap rendah. Untuk skala besar, strategi diferensialmemfokuskan mutasi hanya pada file yang berubah dalam pull request. Pendekatan incremental memungkinkan tim menetapkan target skor per modul alih-alih mengharuskan 100% di seluruh codebase sekaligus.

Scale via differential focus and incremental targets, not full sweeps.

Validasi Race-Clean dan Integritas AST

Bukti teknis bahwa executor konkuren bebas race condition.

TEST RACE

Perintah: `go test -race ./...`
Status: ALL PASSED.
Tidak ada report data race pada stdout/stderr.

MEKANISME KEAMANAN

Setiap goroutine membuat `token.NewFileSet()` baru.
Mutable node hanya diakses dalam goroutine yang sama.
Closure undo mengembalikan state sebelum goroutine selesai.

INDIKATOR KESELAMATAN

Penulisan `results[idx]` aman karena indeks unik per goroutine.
Tidak ada shared pointer atau global mutable state.

Keamanan konkurensi divisualisasikan melalui race detector Go. Karena setiap goroutine bekerja dengan salinan AST baru dan menulis hasil ke indeks slice yang unik, tidak ada kompetisi akses memori. Fungsi undo menjamin tidak ada state AST yang bocor antar goroutine. Validasi ini penting karena mesin mutation testing bergantung pada paralelisasi untuk menjaga latency tetap rendah meskipun jumlah mutan bertambah.

Isolated parses plus indexed writes equal race-free mutation execution.

Checklist Implementasi Mutation Testing

Langkah praktis mengadopsi praktik ini ke dalam pipeline.

PERSIAPAN

[ ] Identifikasi modul domain kritikal.
[ ] Pastikan test suite dapat dieksekusi sebagai `TestFunc`.
[ ] Siapkan AST mutator dengan operator relasional, boolean, aritmetika, boundary.

EKSEKUSI

[ ] Jalankan mutation scan pada file target.
[ ] Evaluasi weak suite sebagai baseline.
[ ] Evaluasi strong suite dengan asersi eksak.

VISUALISASI & REVIEW

[ ] Generate laporan breakdown per mutant.
[ ] Review survived mutants bersama tim.
[ ] Tambah asersi yang menutupi pola survival.

INTEGRASI CI/CD

[ ] Gunakan differential scanning pada PR.
[ ] Tetapkan gate minimum mutation score per modul.

Adopsi mutation testing memerlukan checklist terstruktur agar tidak overwhelming. Dimulai dari identifikasi modul kritikal, persiapan evaluator AST, hingga eksekusi baseline weak dan strong. Tahap visualisasi mendorong review kolektif atas survived mutant. Integrasi CI/CD sebaiknya menggunakan pendekatan diferensial untuk menjaga latency pipeline tetap wajar sambil menegakkan gate skor minimum per modul.

Structured rollout prevents mutation testing fatigue.

Anti-Pattern yang Menciptakan Survived Mutants

Pola berbahaya dalam penulisan test yang perlu dihindari.

ASSERTION POSITIVITAS SAJA

Pola: `assert result > 0` atau `assert !error`.
Dampak: semua mutasi arithmetic dan relasional tetap lolos.
Reproduksi: ubah `>=` jadi `>` pada threshold; test tetap hijau.

INPUT HANYA HAPPY PATH

Pola: menggunakan input jauh dari boundary.
Dampak: boundary shift mutant tidak terdeteksi.
Reproduksi: shift integer 2 ke 3 dengan ItemCount = 5.

TIDAK MENGGUNAKAN EXACT VALUE

Pola: menghindari `assert.Equal(expected, got)`.
Dampak: arithmetic mutation tidak mengubah outcome assert.
Reproduksi: ganti `*` dengan `/` pada amount besar; hasil tetap 'positive'.

Tiga anti-pattern dominan ditemukan pada suite lemah. Assertion berbasis positivitas saja mengabaikan nilai eksak. Input happy path saja melewatkan boundary shift. Ketidakhadiran exact value assertion membuat arithmetic mutation invisibel. Menghilangkan pola-pola ini adalah prasyarat untuk mencapai mutation score yang bermakna.

Positivity assertions and happy-path inputs are mutant shelters.

Takeaway Praktis dan Command Block

Langkah nyata meningkatkan kualitas tes hari ini juga.

```
go test -v ./...
go test -race -v ./...
go run ./cmd/demo
```

ACTIONABLE STEPS

Ganti assertion signe-based dengan exact value/boolean check.
Tambahkan test case di setiap threshold boundary.
Jalankan mutation scan diferensial sebelum merge PR.

MINDSET SHIFT

Coverage hijau bukan jaminan bebas bug.
Mutation score adalah proksi kedalaman asersi.
Setiap survived mutant adalah bug potensial yang menunggu.

Langkah praktis dimulai dengan mengganti asersi longgar menjadi pengecekan exact value dan boolean flag. Tambah test caseboundary di setiap ambang tipe tier dan jumlah item. Integrasikan mutation scan diferensial ke dalam pipeline PR sebagai gate tambahan. Mindset yang berubah adalah memandang coverage sebagai metrik eksekusi, bukan metrik verifikasi, serta看待 survived mutant sebagai indikator bug latent.

Exact assertions and boundary tests convert coverage into verification.

Prinsip Arsitektural Jangka Panjang

Mutation testing sebagai disiplin verifikasi, bukan sekadar metrik.

DISIPLIN VERIFIKASI

Coverage menjawab 'berapa banyak kode dieksekusi'.
Mutation scoring menjawab 'seberapa dalam kode diverifikasi'.
Keduanya komplementer, bukan substitusi.

INCREMENTAL QUALITY GATE

Tetapkan target skor minimum per modul seiring waktu.
Fokus pada modul yang mengalami perubahan di setiap PR.
Hindari perfektisisme global; ukur kemajuan sektoral.

CULTURE IMPROVEMENT

Review survived mutant sebagai sesi pembelajaran tim.
Document pola asersi kuat sebagai reference.
Update checklist anti-pattern secara berkala.

Prinsip arsitektural jangka panjang menempatkan mutation testing sebagai disiplin verifikasi berlapis. Coverage tetap berguna sebagai indikator eksekusi, tetapi mutation score mengukur kedalaman asersi. Pendekatan incremental mencegah kelelahan tim sekaligus mempertahankan tekanan kualitas pada modul kritikal. Budaya review survived mutant mengubah metrik menjadi masukan perbaikan berkelanjutan.

Complementary metrics, incremental gates, cultural review loops.

Mutation Testing: dari Ilusi Coverage menuju Verifikasi Nyata

Verify execution depth, not just execution breadth.

Software Engineering Lab 38 – Mutation Testing

internal/engine/mutator.go • internal/engine/runner.go
internal/service/discount.go • discount_weak_test.go • discount_strong_test.go
tests/engine_test.go • cmd/demo/main.go

REPOSITORY

`github.com/lukman-ss/software-engineering-lab`
`labs/38-mutation-testing`

Lukman / lukman-ss

Sumber: commit 01b1331d
Test passing dan race-clean.
Mutation score kontras: 0.00% vs 100.00%.
Coverage ≠ quality; assertion depth決定 fault detection.
Weak positivity assertions shelter every syntactic fault.
Exact boundary and boolean checks kill mutants reliably.